M07 - PostgreSQL + Data Modeling

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

Primary teaching rule

The SQL scripts are lab assets; the Learner Book explains why each object/rule exists before you run it.

Contents

DE07-01 - Verify PostgreSQL connection and database/schema context

What you will be able to do

Prove which PostgreSQL server, database, user and schema context your query is actually using before changing anything.

Why this matters

A correct SQL statement against the wrong database/schema is still a dangerous mistake. Data engineers verify connection context first.

Picture it this way

Before editing a document, confirm which file and folder are open. PostgreSQL has server -> database -> schema -> table layers; context matters.

Start from zero

- Reuse the PostgreSQL installation from earlier modules. Do not reinstall because M07 starts a database module.
- A PostgreSQL server/cluster can contain multiple databases. A connection is to one database at a time.
- Inside a database, schemas are namespaces containing tables/views/sequences and other objects.
- **current_database()** and **current_user** identify connection context; **current_schema()** and **SHOW search_path** explain unqualified object lookup.
- Use schema-qualified names such as source.orders and warehouse.fact_sales in this module.
- Create a dedicated practice database **stage07_de**. Do not run DROP SCHEMA scripts in another database.

Teacher demonstration

```
SELECT current_database() AS db,
       current_user AS db_user,
       current_schema() AS current_schema;
SHOW search_path;
SELECT version();
```

Read the SQL/step like English

- current_database should be stage07_de when doing the lab.
- current_user is the login role used by pgAdmin/psql.
- search_path shows which schemas PostgreSQL searches for unqualified names.
- version() confirms the server version without changing anything.

Expected check

check	expected
database	stage07_de
query changes	none; read-only verification
object naming	use source.* / warehouse.*

Common beginner traps

- Running setup in postgres/default database because the Query Tool was attached there.
- Dropping/recreating schemas before checking current_database().
- Assuming pgAdmin's tree selection and active Query Tool connection are always the same.

Now you do a similar task

Open a Query Tool connected to stage07_de. Save current_database/current_user/search_path/version results before running any setup script.

How to prove it

- Save SQL/script used.
- Save result/error/EXPLAIN evidence.
- State table/result grain where modeling is involved.
- Reconcile counts/totals rather than trusting plausibility.

STOP - check your understanding

Why is checking current_database() a safety step rather than useless ceremony?

Answer in your own words before continuing.

DE07-02 - Tables, data types and explicit DDL

What you will be able to do

Design table columns with explicit PostgreSQL types/defaults instead of using text for everything.

Why this matters

Types are part of the contract. They prevent impossible operations and communicate the shape/range of stored data.

Picture it this way

A column type is the shape of a storage slot: a date slot is designed for dates, numeric for arithmetic values, boolean for true/false.

Start from zero

- **text** stores variable-length text; **integer/bigint** store whole numbers; **numeric(p,s)** stores exact decimal values useful for money-like calculations.
- **date** stores a calendar date; **timestamptz** stores a timestamp with time-zone-aware semantics.
- **boolean** stores true/false.
- **DEFAULT** supplies a value when an INSERT omits that column, but does not override an explicitly supplied NULL unless constraints/rules say otherwise.
- DDL means Data Definition Language: CREATE/ALTER/DROP define structure.
- Choose a type based on intended meaning and operations, not just what current sample strings look like.

Teacher demonstration

```
CREATE TABLE source.suppliers (
    supplier_id   text PRIMARY KEY,
    supplier_name text NOT NULL,
    country       text NOT NULL,
    active        boolean NOT NULL DEFAULT true,
    rating        numeric(3,2)
                  CHECK (rating BETWEEN 0 AND 5)
);
```

Read the SQL/step like English

- supplier_id is a text business identifier and primary key.
- supplier_name/country are required.
- active defaults to true only when omitted.
- numeric(3,2) can represent values such as 4.75 and CHECK limits business range.

Expected check

column	purpose
supplier_id	row identity
active	boolean with default true
rating	exact decimal, allowed 0..5

Common beginner traps

- Using text for dates/numbers because it avoids conversion work.
- Choosing numeric precision without thinking about allowed maximum.
- Believing DEFAULT automatically fills every NULL.

Now you do a similar task

Create source.suppliers in 05_QUERY_WORKSPACE_TEACHING_RC2.sql with the stated columns. Inspect its definition in pgAdmin or information_schema.columns.

How to prove it

- Save SQL/script used.
- Save result/error/EXPLAIN evidence.
- State table/result grain where modeling is involved.
- Reconcile counts/totals rather than trusting plausibility.

STOP - check your understanding

Why is storing a real date as text weaker than using a date column?

Answer in your own words before continuing.

DE07-03 - Primary keys, foreign keys and constraints

What you will be able to do

Use database constraints to stop invalid identity, relationships and ranges even if application code is wrong.

Why this matters

Validation close to the data protects every writer, not only one Python script.

Picture it this way

Application validation is a receptionist checking forms; database constraints are locked gates that still refuse impossible states.

Start from zero

- **PRIMARY KEY** uniquely identifies a row and disallows NULL.
- **UNIQUE** enforces uniqueness where the column is an alternate key rather than row identity.
- **FOREIGN KEY** requires referenced parent values to exist, protecting referential integrity.
- **NOT NULL** requires a value.
- **CHECK** enforces a boolean condition such as qty > 0 or rating between 0 and 5.
- Constraint failures are expected evidence during testing. Do not 'fix' a failing test by deleting the constraint.
- Constraints do not replace every business rule; they enforce the rules representable and appropriate at the database boundary.

Teacher demonstration

```
INSERT INTO source.suppliers
(supplier_id, supplier_name, country, rating)
VALUES ('S001','Demo Supplier','IN',4.50);
-- Deliberate failure tests, run one at a time:
-- duplicate primary key:
INSERT INTO source.suppliers
VALUES ('S001','Duplicate','IN',true,4.00);
-- invalid rating:
INSERT INTO source.suppliers
VALUES ('S002','Bad Rating','IN',true,6.00);
```

Read the SQL/step like English

- The first insert should succeed.
- The second should fail because supplier_id S001 already exists.
- The third should fail the rating CHECK.
- The errors prove the database is enforcing the model.

Expected check

test	expected
valid supplier	INSERT succeeds
duplicate S001	PK/unique violation
rating 6	CHECK violation

Common beginner traps

- Removing a constraint because bad test data cannot insert.
- Catching every error in application code and assuming DB integrity no longer matters.
- Creating a foreign key to a non-unique/non-key meaning.

Now you do a similar task

Run one valid supplier insert, then deliberately test duplicate primary key, NULL supplier_name and rating 6. Save each error message and explain which rule rejected it.

How to prove it

- Save SQL/script used.
- Save result/error/EXPLAIN evidence.
- State table/result grain where modeling is involved.
- Reconcile counts/totals rather than trusting plausibility.

STOP - check your understanding

What protection does a foreign key add beyond 'we expect these IDs to match'?

Answer in your own words before continuing.

DE07-04 - Transactions, DML and UPSERT

What you will be able to do

Group related writes in transactions, prove rollback/commit behavior and use UPSERT for controlled same-key changes.

Why this matters

Partial updates can create inconsistent data. Transactions make a set of statements succeed/fail as one unit according to the database's transactional rules.

Picture it this way

A transaction is a shopping basket at checkout: either the whole intended purchase is committed or you can abandon the basket before finalizing.

Start from zero

- **BEGIN** starts an explicit transaction; **COMMIT** makes it durable; **ROLLBACK** undoes uncommitted changes.
- Inside a transaction, query your changed value before rollback to prove the temporary state really existed.
- An **UPSERT** combines insert with a defined action on conflict, commonly ON CONFLICT (...) DO UPDATE/NOTHING.
- The conflict target should match a meaningful unique/primary key.
- Transactions do not automatically make bad business logic correct; constraints/validation still matter.
- Be careful with auto-commit behavior in GUI tools: run transaction blocks as intended and inspect results.

Teacher demonstration

```
BEGIN;
UPDATE source.customers
SET region='South', updated_at=now()
WHERE customer_id='C001';
SELECT customer_id, region
FROM source.customers
WHERE customer_id='C001';
ROLLBACK;
SELECT customer_id, region
FROM source.customers
WHERE customer_id='C001';
```

Read the SQL/step like English

- C001 starts in East in the supplied setup.
- Inside the transaction it temporarily shows South.
- ROLLBACK discards that uncommitted update.
- After rollback it should show East again.

Expected check

moment	C001 region
before	East
inside transaction	South
after ROLLBACK	East

Common beginner traps

- Assuming rollback can undo a change after it was already committed.
- Running transaction blocks piecemeal without knowing pgAdmin auto-commit/context.
- Using ON CONFLICT without a business rule for which attributes should update.

Now you do a similar task

Use 02_TRANSACTIONS_UPSERT_TEACHING_RC2.sql. Perform the rollback demo, then one C002 UPSERT. Save before/inside/after evidence.

How to prove it

- Save SQL/script used.
- Save result/error/EXPLAIN evidence.
- State table/result grain where modeling is involved.
- Reconcile counts/totals rather than trusting plausibility.

STOP - check your understanding

What exact boundary separates changes ROLLBACK can undo from changes it cannot?

Answer in your own words before continuing.

DE07-05 - Indexes and EXPLAIN evidence

What you will be able to do

Propose indexes from real query patterns and read EXPLAIN evidence without demanding an Index Scan.

Why this matters

Indexes trade storage/write cost for potential read efficiency. Tiny training tables often favor sequential scans, so evidence matters more than superstition.

Picture it this way

A book index helps when the book is large and the lookup is selective. For a 2-page booklet, reading every line may be cheaper.

Start from zero

- An index is a separate data structure maintained as rows change.
- Useful keys come from actual filter/join/order patterns, not 'index every column'.
- **EXPLAIN** shows the planner's estimated plan without executing a SELECT for actual results.
- **EXPLAIN ANALYZE** executes the statement and adds actual timing/row evidence; use carefully, especially for modifying statements.
- A small table may correctly use Seq Scan before and after index creation.
- Compare plans/row estimates/runtime on representative data before claiming speedup.

Teacher demonstration

```
EXPLAIN
SELECT *
FROM source.orders
WHERE customer_id='C003';
CREATE INDEX IF NOT EXISTS idx_orders_customer_id
ON source.orders(customer_id);
EXPLAIN
SELECT *
FROM source.orders
WHERE customer_id='C003';
```

Read the SQL/step like English

- The query filters on customer_id, so the proposed index matches an access pattern.
- PostgreSQL may still prefer Seq Scan because source.orders has only 8 rows.
- That is not a failed lab. Record the observed plan, not the plan you wanted.

Expected check

evidence	expected behavior
before plan	record it
after plan	record it
tiny table	Seq Scan is acceptable
claim	only what evidence supports

Common beginner traps

- Saying index made it faster without measurement.
- Creating indexes on every field.
- Treating Seq Scan as an error.

Now you do a similar task

Run the before/after EXPLAIN from 03_INDEX_EXPLAIN_TEACHING_RC2.sql. Write what changed or did not, and why a tiny table may choose Seq Scan.

How to prove it

- Save SQL/script used.
- Save result/error/EXPLAIN evidence.
- State table/result grain where modeling is involved.
- Reconcile counts/totals rather than trusting plausibility.

STOP - check your understanding

Why can Seq Scan be the correct plan even after creating a logically useful index?

Answer in your own words before continuing.

DE07-06 - Operational vs analytical modeling and normalization

What you will be able to do

Understand why transactional/operational models often normalize data and analytical models reshape it for querying.

Why this matters

Data modeling is choosing where facts/attributes live so updates and queries remain trustworthy.

Picture it this way

An operational system is a well-organized filing cabinet that avoids writing the customer's address on every line item. An analytical model is a prepared map designed to answer reporting questions quickly and consistently.

Start from zero

- The **operational/source model** in this lab has customers, products, orders and order_items separated by business entities/relationships.
- **Normalization** reduces unnecessary repetition and update anomalies by placing facts/attributes in appropriate related tables.
- If customer region were copied into every order line, a region change would require many updates and could leave contradictory rows.
- Analytical models often intentionally denormalize/reorganize data into facts/dimensions for easier consistent aggregation.
- Normalization is not a moral score. Model based on workload, integrity and ownership.
- Always define business keys and row grain before deciding table boundaries.

Teacher demonstration

```
-- Operational relationship:
source.customers
  1 customer row
source.orders
  many orders reference customer_id
source.order_items
  many lines reference order_id
-- Customer name/region are not copied into every order line.
```

Read the SQL/step like English

- Customer attributes have one authoritative source row in the operational design.
- Orders reference the customer key instead of duplicating name/region.
- Order items reference order + product and store line-specific measures.
- This reduces update anomalies in the source model.

Expected check

model	main goal
operational normalized	reliable writes / entity integrity
analytical star	consistent/simple analytical queries

Common beginner traps

- Calling every wide table 'denormalized and therefore bad'.
- Normalizing analytical output so aggressively that simple reports require dozens of joins without benefit.
- Modeling before deciding row grain/business key.

Now you do a similar task

Write three concrete problems caused by copying customer_name and region into every source.order_items row. Then explain why an analytical star schema may still repeat keys across fact rows.

How to prove it

- Save SQL/script used.
- Save result/error/EXPLAIN evidence.
- State table/result grain where modeling is involved.
- Reconcile counts/totals rather than trusting plausibility.

STOP - check your understanding

Why can reducing duplication help operational integrity but a star schema still intentionally repeat dimension keys in fact rows?

Answer in your own words before continuing.

DE07-07 - Facts, dimensions and star-schema grain

What you will be able to do

Define a fact-table grain first, then understand dimensions, measures and a star-schema query.

Why this matters

A fact table without a clear grain creates double-counting and ambiguous measures.

Picture it this way

Grain answers: 'What does one row mean?' Before counting money, everyone must agree whether one row is an order, order line, day, customer-day, etc.

Start from zero

- A **fact table** stores measurable events at a declared grain plus dimension foreign keys.
- A **dimension** stores descriptive context such as customer/product/date attributes.
- The supplied **warehouse.fact_sales grain is ONE ROW PER COMPLETED ORDER LINE**.
- Because it is line grain, one order may legitimately have multiple fact rows.
- **gross_sales = qty * unit_price** is additive across these independent line rows in this toy model.
- Cancelled O1005 must produce zero fact rows because the ETL business rule includes Completed orders only.

Teacher demonstration

```
SELECT c.region,
       SUM(f.gross_sales) AS revenue
FROM warehouse.fact_sales f
JOIN warehouse.dim_customer c
  ON c.customer_key=f.customer_key
GROUP BY c.region
ORDER BY c.region;
```

Read the SQL/step like English

- fact_sales contributes line-level measures.
- dim_customer converts surrogate customer_key into descriptive region.
- GROUP BY region changes result grain to one row per region.
- Regional totals should reconcile to overall fact revenue 14130.00.

Expected check

region	revenue
East	3710.00
North	2390.00
South	1750.00
West	6280.00
TOTAL	14130.00

Common beginner traps

- Calling fact_sales one row per order when orders can have several lines.
- Summing order-level measures repeated across line-level rows.
- Skipping total reconciliation after grouping.

Now you do a similar task

Write the fact_sales grain in one sentence. Query revenue by region and by product category. Reconcile each grouped result back to total revenue.

How to prove it

- Save SQL/script used.
- Save result/error/EXPLAIN evidence.
- State table/result grain where modeling is involved.
- Reconcile counts/totals rather than trusting plausibility.

STOP - check your understanding

If an order has two product lines, how many fact_sales rows should it create under this model, and why?

Answer in your own words before continuing.

DE07-08 - Surrogate keys and referential integrity

What you will be able to do

Distinguish business/natural keys from surrogate warehouse keys and prove referential integrity.

Why this matters

Warehouse dimensions need stable internal references while preserving source business identifiers.

Picture it this way

customer_id C003 is the business's ID printed on the customer's card. customer_key 17 is the warehouse's internal drawer number used to connect facts to the correct dimension row.

Start from zero

- A **natural/business key** comes from business/source meaning, such as customer_id.
- A **surrogate key** is generated internally, commonly an integer/identity, with no business meaning by itself.
- dim_customer keeps customer_id as a unique alternate key and customer_key as the primary surrogate key.
- fact_sales stores customer_key/product_key so facts reference warehouse dimension rows.
- Foreign keys reject nonexistent surrogate references.
- For SCD history, surrogate keys become especially useful because one business customer_id can have multiple historical dimension versions.

Teacher demonstration

```
SELECT customer_key, customer_id,
       customer_name, region
FROM warehouse.dim_customer
ORDER BY customer_key;
SELECT f.order_id, f.line_no,
       f.customer_key, c.customer_id
FROM warehouse.fact_sales f
JOIN warehouse.dim_customer c
  ON c.customer_key=f.customer_key
ORDER BY f.order_id, f.line_no
LIMIT 5;
```

Read the SQL/step like English

- The first query shows internal key beside business key.
- The second proves fact->dimension referential relationship.
- Facts do not need to duplicate customer_name/region to identify the warehouse customer row.

Expected check

integrity check	expected
orphan customer FKs	0
orphan product FKs	0
business customer_id	unique in Type-1 dim_customer

Common beginner traps

- Treating surrogate key as a business identifier users should type/remember.
- Dropping the natural key entirely and losing source traceability.
- Using dimension name text as fact foreign key.

Now you do a similar task

Display customer_id/customer_key pairs. In a transaction, deliberately attempt a fact insert with a nonexistent product_key, observe the FK error and roll back.

How to prove it

- Save SQL/script used.
- Save result/error/EXPLAIN evidence.
- State table/result grain where modeling is involved.
- Reconcile counts/totals rather than trusting plausibility.

STOP - check your understanding

Why can a surrogate key become more important once one business customer has multiple historical dimension versions?

Answer in your own words before continuing.

DE07-09 - Slowly Changing Dimension Type 2 history

What you will be able to do

Preserve attribute history with a simple SCD Type 2 pattern instead of overwriting the past.

Why this matters

Analysts may need to know what a customer's region/segment was at the time of an event, not only the latest value.

Picture it this way

Instead of erasing an old address on a form, close the old record's valid-to date and add a new dated version.

Start from zero

- SCD means **Slowly Changing Dimension**. Type 2 keeps multiple rows for historical versions.
- Typical fields include effective_from, effective_to and is_current.
- One business key customer_id can have multiple surrogate customer_key rows over time.
- To apply a change: close current row's date range/is_current, then insert a new current version.
- The toy lab uses a CHECK tying current rows to NULL effective_to.
- Real production SCD2 needs stronger protection against overlapping ranges/multiple current rows, often via pipeline rules and additional constraints/indexes.

Teacher demonstration

```
-- C003 moves North -> East effective 2026-09-01
BEGIN;
UPDATE warehouse.dim_customer_scd2
SET effective_to='2026-08-31',
    is_current=false
WHERE customer_id='C003'
  AND is_current=true;
INSERT INTO warehouse.dim_customer_scd2
(customer_id,customer_name,region,segment,
 effective_from,effective_to,is_current)
SELECT customer_id,customer_name,'East',segment,
       '2026-09-01',NULL,true
FROM source.customers
WHERE customer_id='C003';
COMMIT;
```

Read the SQL/step like English

- The old current C003 version is closed through 2026-08-31.
- A new East version begins 2026-09-01.
- The history now contains two C003 rows, one old/non-current and one new/current.
- The transaction keeps the two-step state change together.

Expected check

C003 version	region/current
2026-08-01..2026-08-31	North / false
2026-09-01..open	East / true
current rows for C003	exactly 1

Common beginner traps

- Updating region in place and destroying historical truth.
- Creating a new current row but forgetting to close old current row.
- Assuming this toy table fully prevents overlapping date ranges in every production case.

Now you do a similar task

Run 04_SCD2_LAB_TEACHING_RC2.sql and explain the two C003 rows. Then sketch how a future segment change would add another version without rewriting history.

How to prove it

- Save SQL/script used.
- Save result/error/EXPLAIN evidence.
- State table/result grain where modeling is involved.
- Reconcile counts/totals rather than trusting plausibility.

STOP - check your understanding

What information is lost if you simply UPDATE the old C003 region from North to East?

Answer in your own words before continuing.

DE07-10 - Build and validate a small analytical database

What you will be able to do

Build the supplied source + star schema and prove row counts, grain, referential integrity and revenue reconciliation.

Why this matters

A database build is not finished when CREATE/INSERT statements run without errors. You need model-level validation.

Picture it this way

Building a warehouse is not complete because shelves exist; inventory counts, labels and references must reconcile.

Start from zero

- The setup builds **source** operational tables and **warehouse** dimensions/fact.
- Fact load includes only Completed orders and uses one row per completed order line.
- Expected fact rows = 12; total qty = 30; total revenue = 14130.00.
- Cancelled O1005 must have zero rows in fact_sales.
- UNIQUE(order_id,line_no) protects fact grain from duplicate loads.
- Foreign-key orphan checks must return zero.
- Regional totals must sum to the same overall revenue.

Teacher demonstration

```
SELECT COUNT(*) AS fact_rows,
       SUM(qty) AS total_qty,
       SUM(gross_sales) AS total_revenue
FROM warehouse.fact_sales;
SELECT COUNT(*) AS cancelled_fact_rows
FROM warehouse.fact_sales
WHERE order_id='O1005';
```

Read the SQL/step like English

- First query checks three high-level control totals.
- Second checks a business exclusion rule.
- Additional validation script checks duplicate grain, orphans and regional reconciliation.

Expected check

check	expected
fact rows	12
total qty	30
total revenue	14130.00
O1005 fact rows	0
orphan dimension refs	0

Common beginner traps

- Declaring success because setup script ended.
- Checking row count but not revenue/business exclusions.
- Using a final dashboard result as the only validation evidence.

Now you do a similar task

Run 01_SETUP_AND_BUILD_TEACHING_RC2.sql, then run 06_FINAL_VALIDATION_TEACHING_RC2.sql. Explain every validation query in plain English.

How to prove it

- Save SQL/script used.
- Save result/error/EXPLAIN evidence.
- State table/result grain where modeling is involved.
- Reconcile counts/totals rather than trusting plausibility.

STOP - check your understanding

Why are 12 fact rows and 14130 revenue both needed as evidence instead of only one of them?

Answer in your own words before continuing.

DE07-11 - Bulk loading with COPY / staging boundaries

What you will be able to do

Understand controlled bulk loading into staging and the difference between PostgreSQL COPY, psql \copy and transformation into final tables.

Why this matters

Real pipelines often land files into staging before enforcing final relationships/transformations.

Picture it this way

Bulk load is unloading a truck into a receiving area first. You inspect and reconcile the shipment before moving items to permanent shelves.

Start from zero

- PostgreSQL **COPY** is a server SQL command; file paths are interpreted from the database server's environment/permissions.
- psql **\copy** is a psql client command that reads a file from the client machine and sends rows to the server.
- pgAdmin also offers Import/Export UI. Pick a tool you can explain instead of mixing syntaxes.
- Load into a **staging table** when raw file types/quality need inspection before final constrained tables.
- Count staging rows and compare with source file rows before transformation.
- Do not treat CSV header as a business row; use HEADER option/UI equivalent.
- After successful validation/insert to final, decide archive/quarantine policy outside the database as part of the pipeline.

Teacher demonstration

```
-- In PostgreSQL:
CREATE TABLE source.stg_customers (
    customer_id text,
    customer_name text,
    region text,
    segment text
);
-- If using psql from a terminal, client-side example:
-- \copy source.stg_customers
-- FROM 'customers.csv'
-- WITH (FORMAT csv, HEADER true);
SELECT COUNT(*) FROM source.stg_customers;
```

Read the SQL/step like English

- Staging columns are deliberately permissive text in this simple receiving layer.
- \copy is NOT plain SQL and works in psql, not pgAdmin Query Tool.
- If you use pgAdmin Import/Export instead, the validation target is the same staging table/count.
- Only after row/schema checks should data move into final constrained tables.

Expected check

concept	correct interpretation
COPY	server-side file access
\copy	psql client-side file access
staging	receive/inspect before final model

concept	correct interpretation
validation	file/business row count reconciles

Common beginner traps

- Pasting \copy into pgAdmin Query Tool and assuming PostgreSQL SQL syntax is broken.
- Loading raw CSV straight into final tables when validation/transformation is required.
- Ignoring header/encoding/delimiter settings.

Now you do a similar task

Load one provided CSV into a staging table using either psql \copy or pgAdmin Import/Export. Save source business-row count and staging COUNT(*), then explain why staging is separate from final warehouse tables.

How to prove it

- Save SQL/script used.
- Save result/error/EXPLAIN evidence.
- State table/result grain where modeling is involved.
- Reconcile counts/totals rather than trusting plausibility.

STOP - check your understanding

Why can \copy work in psql but fail as syntax in pgAdmin Query Tool?

Answer in your own words before continuing.

DE07-12 - Schemas, roles and least-privilege boundaries

What you will be able to do

Use schemas/roles to create clearer ownership and least-privilege access boundaries.

Why this matters

Production databases should not give every application/user ownership or unrestricted write access to every table.

Picture it this way

Schemas are rooms; roles are key rings. Give each role only the room doors and actions it actually needs.

Start from zero

- A PostgreSQL **role** can represent a login/user or a group-like privilege container.
- Privileges include CONNECT, USAGE on schema, SELECT/INSERT/UPDATE/DELETE on tables, EXECUTE, etc.
- **least privilege** means grant only what a workload needs.
- A reporting reader may need USAGE on warehouse schema + SELECT on warehouse tables, not UPDATE/DROP.
- Object ownership is stronger than ordinary privileges; avoid using the owner/superuser for application/reporting connections.
- Role creation/grants require sufficient privileges. If your local account cannot create roles, complete the design/explain portion rather than changing unrelated system settings.

Teacher demonstration

```
-- Run only in your local practice environment if your account permits:
CREATE ROLE stage07_reader NOLOGIN;
GRANT USAGE ON SCHEMA warehouse
TO stage07_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA warehouse
TO stage07_reader;
-- Inspect:
SELECT grantee, table_schema, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE grantee='stage07_reader'
ORDER BY table_schema, table_name, privilege_type;
```

Read the SQL/step like English

- NOLOGIN makes stage07_reader a privilege container, not a direct password login.
- USAGE allows name resolution/access into the schema subject to object privileges.
- SELECT permits reads on current warehouse tables.
- This does not grant INSERT/UPDATE/DROP.

Expected check

role need	privilege
reporting reader	USAGE warehouse + SELECT
source writes	not needed for reader
DROP/ownership	not granted

Common beginner traps

- Running applications as postgres/superuser because it avoids permission work.
- Granting ALL ON EVERYTHING without a requirement.
- Assuming a role design automatically covers future tables/default privileges.

Now you do a similar task

Design two roles on paper/SQL comments: warehouse_reader and source_loader. State exact schema/table privileges each needs and does not need. If permitted, create only the reader NOLOGIN role locally and inspect grants.

How to prove it

- Save SQL/script used.
- Save result/error/EXPLAIN evidence.
- State table/result grain where modeling is involved.
- Reconcile counts/totals rather than trusting plausibility.

STOP - check your understanding

Why is running a reporting tool as the database owner or superuser risky even if it is convenient?

Answer in your own words before continuing.

DE07-13 - Schema change, backup/restore and handoff validation

What you will be able to do

Change schema deliberately, create a recoverable backup, restore/validate a copy, and produce a database handoff checklist.

Why this matters

Data models evolve. Safe engineering requires change evidence and a recovery path, not ad-hoc manual edits with no validation.

Picture it this way

A schema migration is a controlled building renovation; backup/restore is your tested emergency copy, and handoff is the building manual.

Start from zero

- A schema change should have a purpose, ordered DDL, compatibility/data-migration considerations and validation.
- **ALTER TABLE** changes existing structure; avoid manual GUI-only changes that nobody can reproduce.
- **pg_dump** creates a logical backup. Custom format (-Fc) is commonly restored with pg_restore.
- Do not put database passwords in command-line text/history. Let tools prompt or use appropriate protected credential mechanisms.
- Backup is not proven until you can restore to a separate database/environment and run validation.
- pgAdmin Backup/Restore UI is acceptable if you document the options and validate the restored copy.
- Handoff evidence: schema/build scripts, model grain, row/control totals, role assumptions, backup date/file, restore validation and known limitations.

Teacher demonstration

```
-- Example reversible-looking schema change plan:
ALTER TABLE source.products
ADD COLUMN IF NOT EXISTS active boolean NOT NULL DEFAULT true;
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_schema='source'
  AND table_name='products'
ORDER BY ordinal_position;
-- Terminal backup example (do not embed password):
-- pg_dump -Fc -d stage07_de -f stage07_de.backup
-- Restore to a separate test database, then run final validation.
```

Read the SQL/step like English

- ALTER is reproducible SQL rather than a hidden GUI change.
- The information_schema query proves the column definition.
- Backup command example writes a custom-format logical dump.
- A restored database must pass the same fact count/revenue/orphan checks to count as a successful recovery test.

Expected check

handoff item	evidence
schema change	tracked SQL + definition check
backup	dump file + date/tool/options
restore	separate copy/database
post-restore validation	12 fact rows, qty 30, revenue 14130, no orphans

Common beginner traps

- Calling a backup successful without ever restoring it.
- Saving only screenshots of GUI schema edits with no reproducible SQL.
- Restoring over the original practice database for the first test.

Now you do a similar task

Add the active column in your practice database, validate its definition, make a logical backup with pgAdmin or pg_dump, restore to a separate test database if your environment allows, and run final validation. Record any tool/path issue instead of guessing.

How to prove it

- Save SQL/script used.
- Save result/error/EXPLAIN evidence.
- State table/result grain where modeling is involved.
- Reconcile counts/totals rather than trusting plausibility.

STOP - check your understanding

Why is 'I have a .backup file' weaker evidence than 'I restored it to a separate database and validation passed'?

Answer in your own words before continuing.